

Data Analysis with Python 2024–2025

Parte 3: Pandas (Parte 1)

Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024–2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Saber o que é a biblioteca Pandas.
- Entender os DataFrames.
- Aprender a criar e indexar séries (*Series*).

Conteúdo

1 Pandas (Parte 1)

Na seção do NumPy, lidamos com alguns arrays cujas colunas tinham, cada uma, um significado especial. Por exemplo, a coluna de número 0 poderia conter valores interpretados como anos, e a coluna 1 poderia conter um mês. É possível manipular os dados dessa forma, mas pode ser difícil lembrar qual número de coluna corresponde a qual variável. Especialmente se você remover alguma coluna do array posteriormente, pois a numeração das colunas restantes será alterada. Uma solução para isso é dar um nome descritivo a cada coluna. Esses nomes de coluna permanecem fixos e anexados às suas colunas correspondentes, mesmo que removamos algumas delas. Além disso, as linhas também podem receber nomes; no Pandas, esses nomes são chamados de **índices** (*indices*).

A biblioteca Pandas é construída sobre a biblioteca NumPy e fornece um tipo especial de estrutura de dados bidimensional chamada **DataFrame**. O DataFrame permite dar nomes às colunas, de forma que você possa acessar uma coluna usando seu nome em vez do seu índice numérico.

Primeiro, passaremos rapidamente por alguns exemplos para ver o que é possível fazer com o Pandas. Você pode precisar verificar alguns detalhes na documentação do Pandas para concluir os exercícios. Começamos fazendo algumas importações padrão:

```
import pandas as pd      # Esta é a forma padrão de importar a biblioteca
                          Pandas
import numpy as np
```

Vamos importar alguns dados meteorológicos que estão em formato de texto num arquivo csv (*Comma Separated Values*). A chamada a seguir buscará os dados da internet e os converterá em um DataFrame:

```
wh = pd.read_csv("https://raw.githubusercontent.com/csmastersUH/
data_analysis_with_python_2020/master/kumpula-weather-2017.csv")
wh.head()      # O método head imprime as primeiras 5 linhas
```

	Year	m	d	Time	Time zone	Precipitation amount (mm)	Snow depth (cm)	Air temperature (degC)
0	2017	1	1	00:00	UTC	-1.0	-1.0	0.6
1	2017	1	2	00:00	UTC	4.4	-1.0	-3.9
2	2017	1	3	00:00	UTC	6.6	7.0	-6.5
3	2017	1	4	00:00	UTC	-1.0	13.0	-12.8
4	2017	1	5	00:00	UTC	-1.0	10.0	-17.8

Vemos que o DataFrame contém oito colunas, três das quais são variáveis medidas reais. Agora podemos nos referir a uma coluna pelo seu nome:

```
wh["Snow depth (cm)"].head()      # Usar a tecla Tab pode ajudar a digitar
names longos
```

```
0      -1.0
1      -1.0
2       7.0
3      13.0
4      10.0
Name: Snow depth (cm), dtype: float64
```

Existem vários métodos de estatística resumida que operam em uma coluna ou em todas as colunas. O próximo exemplo calcula a média das temperaturas sobre todas as linhas do DataFrame:

```
wh["Air temperature (degC)"].mean()      # Temperatura média
```

6.5271232876712331

Podemos remover algumas colunas do DataFrame com o método `drop`:

```
wh.drop("Time zone", axis=1).head() # Retorna uma copia com uma coluna
removida, o DataFrame original permanece intacto
```

	Year	m	d	Time	Precipitation amount (mm)	Snow depth (cm)	Air temperature (degC)
0	2017	1	1	00:00	-1.0	-1.0	0.6
1	2017	1	2	00:00	4.4	-1.0	-3.9
2	2017	1	3	00:00	6.6	7.0	-6.5
3	2017	1	4	00:00	-1.0	13.0	-12.8
4	2017	1	5	00:00	-1.0	10.0	-17.8

```
wh.head() # O DataFrame original permanece inalterado
```

Caso você queira modificar o DataFrame original, pode atribuir o resultado ao DataFrame original ou usar o parâmetro `inplace` do método `drop`. Muitos dos métodos de modificação do DataFrame possuem o parâmetro `inplace`.

A adição de uma nova coluna funciona como adicionar um novo par chave-valor a um dicionário (*dictionary*):

```
wh["Rainy"] = wh["Precipitation amount (mm)"] > 5
wh.head()
```

	Year	m	d	Time	Time zone	Precipitation amount (mm)	Snow depth (cm)	Air temperature (degC)	Rainy
0	2017	1	1	00:00	UTC	-1.0	-1.0	0.6	False
1	2017	1	2	00:00	UTC	4.4	-1.0	-3.9	False
2	2017	1	3	00:00	UTC	6.6	7.0	-6.5	True
3	2017	1	4	00:00	UTC	-1.0	13.0	-12.8	False
4	2017	1	5	00:00	UTC	-1.0	10.0	-17.8	False

Nas próximas seções, passaremos sistematicamente pelo DataFrame e sua versão unidimensional: a **Series**.

2 Criação e indexação de séries (*Series*)

Você pode transformar qualquer iterável unidimensional em uma **Series**, que é um array unidimensional com rótulos nos eixos.

```
s = pd.Series([1, 4, 5, 2, 5, 2])
s
```

```
0    1
1    4
2    5
3    2
4    5
5    2
dtype: int64
```

O tipo de dados dos elementos nesta Série é `int64`, ou seja, inteiros representáveis em 64 bits.

Também podemos anexar um nome a esta série:

```
s.name = "Grades"
s
```

```
0    1
1    4
2    5
3    2
4    5
5    2
Name: Grades, dtype: int64
```

Os atributos comuns das séries são o `name`, `dtype` e `size`:

```
print(f"Name: {s.name}, dtype: {s.dtype}, size: {s.size}")
```

```
Name: Grades, dtype: int64, size: 6
```

Além dos valores da série, também foram impressos os índices das linhas. Todos os métodos de acesso de arrays NumPy também funcionam para a `Series`: indexação, *slicing* (fatiamento), *masking* (mascaramento) e *fancy indexing* (indexação avançada).

```
s[1]
```

```
4
```

```
s2 = s[[0, 5]] # Fancy indexing
print(s2)
```

```
0    1
5    2
Name: Grades, dtype: int64
```

```
t = s[-2:] # Slicing
t
```

```
4    5
5    2
Name: Grades, dtype: int64
```

Observe que os índices permanecem vinculados aos valores correspondentes; eles não são reenumerados!

```
t[4]
```

```
5
```

(Nota: `t[0]` daria um erro). Os valores na forma de um array NumPy são acessíveis através do atributo `values`:

```
s2.values
```

```
array([1, 2])
```

E os índices estão disponíveis através do atributo `index`:

```
s2.index
```

```
Int64Index([0, 5], dtype='int64')
```

O índice não é simplesmente um array NumPy, mas uma estrutura de dados que permite acesso rápido aos elementos. Os índices não precisam ser inteiros, como o próximo exemplo mostra:

```
s3 = pd.Series([1, 4, 5, 2, 5, 2], index=list("abcdef"))
s3
```

```
a    1
b    4
c    5
d    2
e    5
f    2
dtype: int64
```

```
s3.index
```

```
Index(['a', 'b', 'c', 'd', 'e', 'f'], dtype='object')
```

```
s3["b"]
```

```
4
```

```
s3["b":"e"]
```

```
b    4
c    5
d    2
e    5
dtype: int64
```

Ainda é possível acessar a série usando índices inteiros implícitos, no estilo NumPy:

```
s3[1]
```

```
4
```

No entanto, isso pode ser confuso. Considere a seguinte série:

```
s4 = pd.Series(["Jack", "Jones", "James"], index=[1, 2, 3])
s4
```

```
1    Jack
2   Jones
3   James
dtype: object
```

O que você acha que `s4[1]` vai imprimir? Para evitar essa ambiguidade, o Pandas oferece os atributos `loc` e `iloc`. O atributo `loc` sempre usa o índice explícito, enquanto o atributo `iloc` sempre usa o índice inteiro implícito:

```
print(s4.loc[1])
print(s4.iloc[1])
```

Jack
Jones

Vale notar que existem similaridades entre os dicionários do Python e as Séries do Pandas; ambos podem ser pensados como formas de acessar valores usando chaves (*keys*). A diferença é que a Série requer que todos os índices tenham o mesmo tipo e, da mesma forma, que todos os valores tenham o mesmo tipo. Essa restrição permite a criação de estruturas de dados rápidas.

Como prova das similaridades entre essas duas estruturas de dados, o Pandas permite a criação de um objeto **Series** a partir de um dicionário:

```
d = {2001: "Bush", 2005: "Bush", 2009: "Obama", 2013: "Obama", 2017: "Trump"}
s4 = pd.Series(d, name="Presidents")
s4
```

```
2001    Bush
2005    Bush
2009    Obama
2013    Obama
2017    Trump
Name: Presidents, dtype: object
```

3 Resumo (Semana 3)

- Você descobriu que comparações também são operações vetorizadas, e que o resultado de uma comparação pode ser usado para mascarar (ou seja, restringir) operações subsequentes em arrays.
- Você pode selecionar uma lista de colunas usando indexação avançada (*fancy indexing*).
- Uma aplicação dos arrays NumPy: operações básicas de álgebra linear e resolução de sistemas de equações lineares.
- Você conhece os blocos de construção das figuras do *matplotlib*. Você pode criar figuras baseadas em arrays NumPy e pode ajustar os atributos das figuras.
- Você entende como imagens (*raster*) são organizadas como arrays NumPy. Você pode manipular imagens usando as operações de arrays do NumPy.
- No Pandas, é padrão usar linhas de **DataFrames** como amostras e colunas como variáveis.
- Tanto linhas quanto colunas dos **DataFrames** do Pandas podem ter nomes, ou seja, índices.
- Operações mantêm esses índices mesmo quando se adicionam ou removem linhas ou colunas.
- Os índices também permitem que várias operações sejam combinadas de forma significativa e fácil.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.